

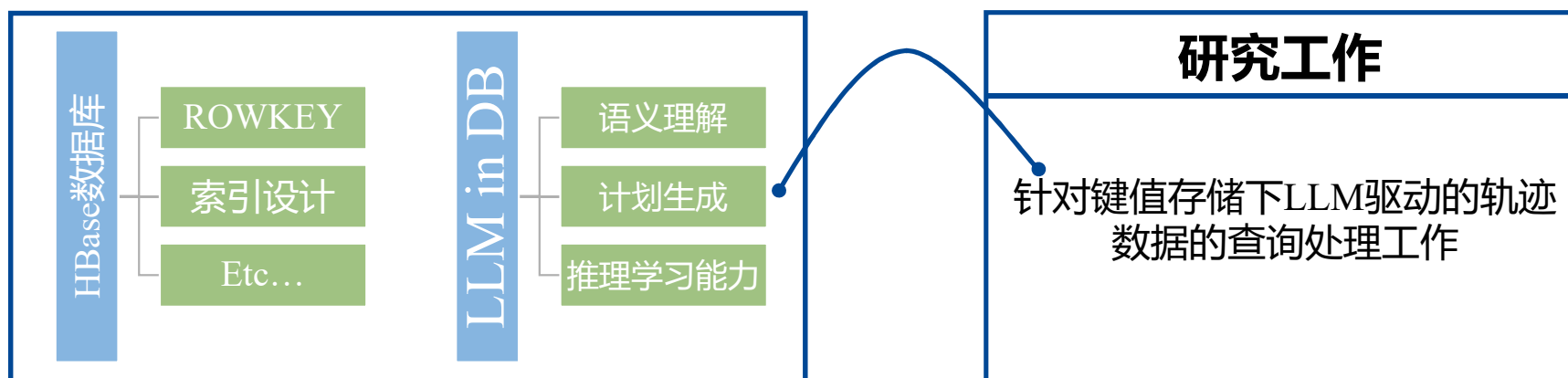


键值存储下基于LLM的轨迹数据查询

研究背景



- **KV数据库使用广泛:** 目前产生的**轨迹数据很多**, 主要**存储在KV数据库**中
- **键值数据库查询依赖数据模型:** 用户查询通常需要了解**数据模型和访问路径的设计**
- **LLM发展迅速:** LLM技术愈发成熟, 适配**多种应用场景**
- **关系型数据库针对LLM的应用广泛:** 目前**LLM**在许多**关系型数据库**中的工作应用广泛, (**NL2SQL**, **查询优化等**)



JT1

如何使用自然语言高效, 准确的在键值数据库下处理轨迹数据查询?

目前对于这个问题现有研究的不足:

- 主要研究使用自然语言在**关系型数据库**下生成**SQL**进行查询

DTS-SQL: Decomposed Text-to-SQL with Small Large Language Models [10.18653/v1/2024.findings-emnlp.481](https://arxiv.org/abs/10.18653/v1/2024.findings-emnlp.481) (Findings of EMNLP 2024)

- 已有自然语言生成**NoSQL**的研究方面并不完善,只有一篇关于MongoDB的工作

Bridging the Gap: Enabling Natural Language Queries for NoSQL Databases through Text-to-NoSQL Translation <https://arxiv.org/abs/2502.11201>

- 目前KV数据库查询优化方面的研究主要集中于: 预先设计**数据布局与索引**, 针对于**查询计划**部分的工作很少 JT2

TFQS: HBase-based Cache Optimization Model for Secondary Index <https://doi.org/10.1109/aeis65978.2024.00013> (AEIS 2024)

幻灯片 3

- JT1** 使用LLM理由: LLM是目前用于处理自然语言理解最好的工具
Jay Tang, 2026/9/19
- JT2** 针对不断变化的应用场景, 重新设计成本高, 查询计划部分优化更适合解决我这个问题
Jay Tang, 2026/9/19

挑战



自然语言查询转换为KV计划困难

- 目前只有转SQL的工作, **难以应用到KV数据库**
- 实际查询计划**难以正确理解**自然语言语义
- 查询条件需要**映射到物理数据结构**
- 自然语言可能**无法正确表意**, 不能正确转化为KV计划

查询优化中选择高效查询路径困难

- 目前针对**KV数据库的查询计划优化部分存在空白**
- **固定规则或者枚举计划**都存在显著缺陷
- **数据分布变化**会使查询规则失效
- LLM完全主导无法很好的**感知Hbase的状态**, 只能依据输入信息判断

保证LLM生成的KV计划正确性困难

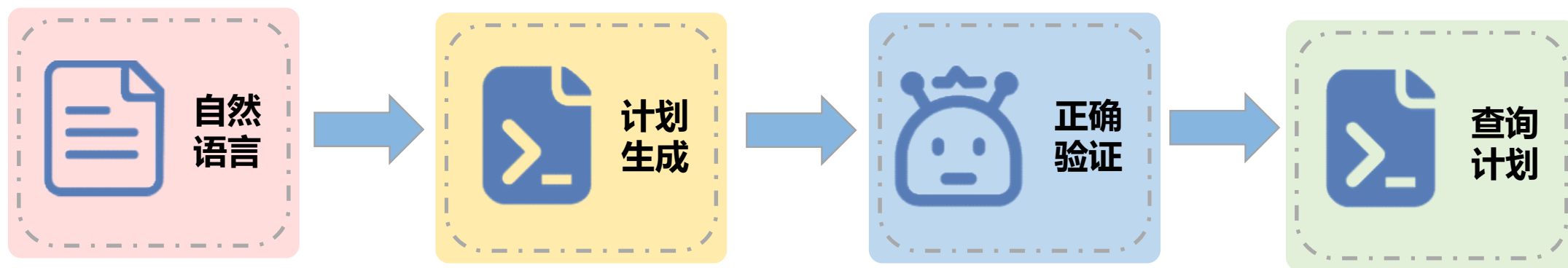
- LLM容易**生成结构合法但是实际查询不正确的计划**
- 索引与轨迹数据可能**更新不及时**
- 即使自然语言被正确理解, 计划也有可能无法被编译为正确的物理计划(逻辑计划→物理计划)

任务定义



任务: 构建一个LLM驱动的键值存储下的自然语言轨迹数据查询系统

- **输入:** 自然查询语言/统计信息
- **输出:** 正确性验证后的针对组合和排序多个索引及候选过滤操作部分优化后的查询计划

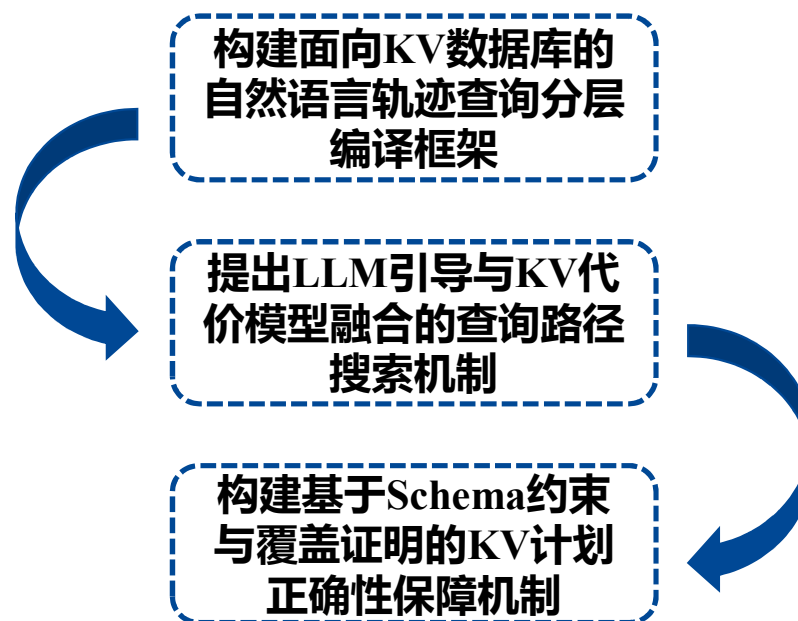


解决方法



- 自然语言查询转换为KV计划困难
- 查询优化中搜索和选择高效访问路径困难
- 保证LLM生成的KV计划正确性困难

对应的解决方法



分层编译框架



语义解析流程



用户: 查询早高峰时期的车辆数



LLM主动提问: 早高峰具体时间是多久? 地点在哪里?



用户: 补充信息

KV逻辑计划



- **受限算子**中进行选择
- 使用哪些**索引**
- **过滤与读取**的顺序
- 何时**精确过滤**

分层编译框架

自然语言



输入自然语言
LLM进行处理



KV逻辑计划

LLM提出逻辑算子组合



确定性
KV编译



编译最终计划
输出可执行KV计划

语义解析

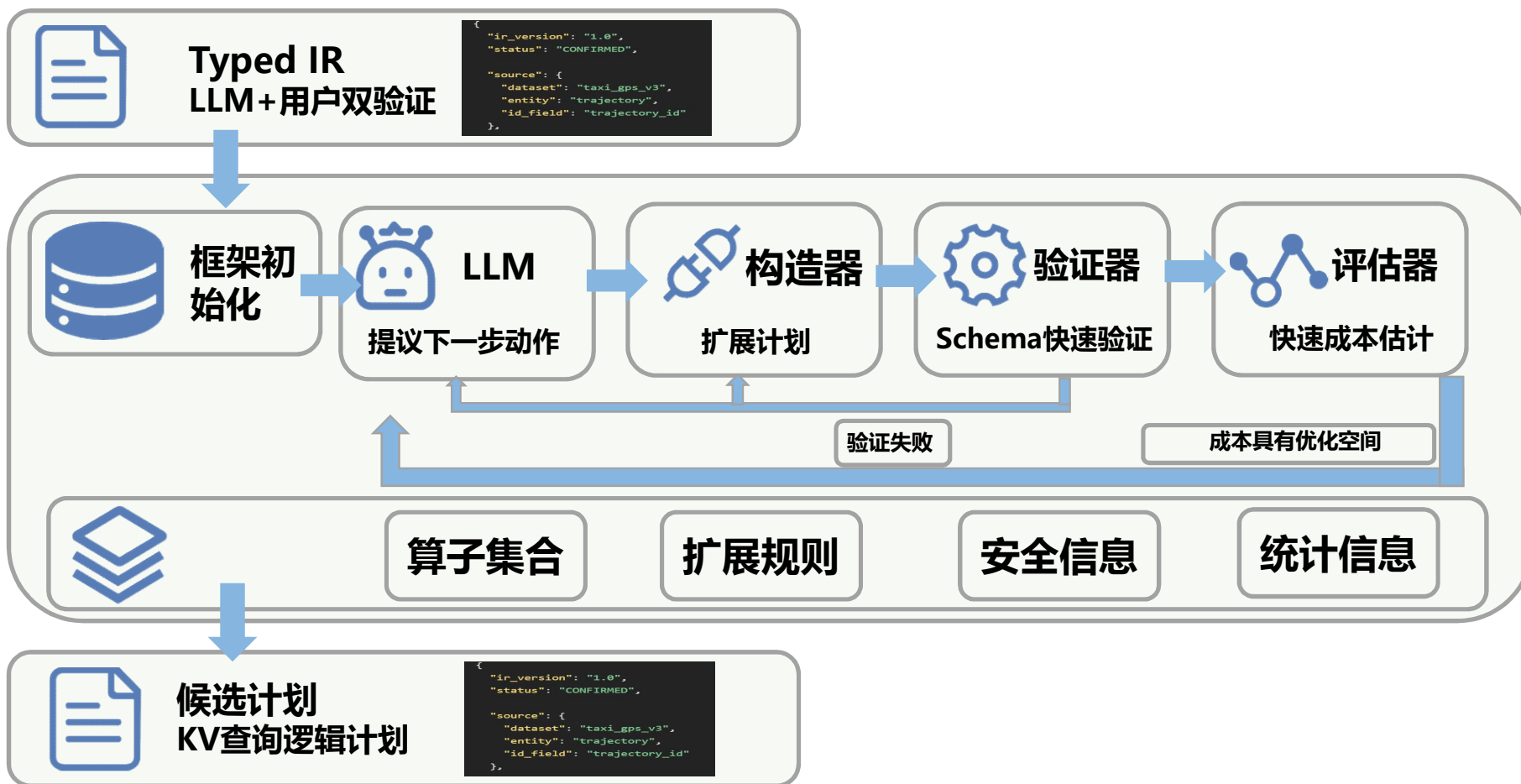
+ 使用 ChatGPT Work



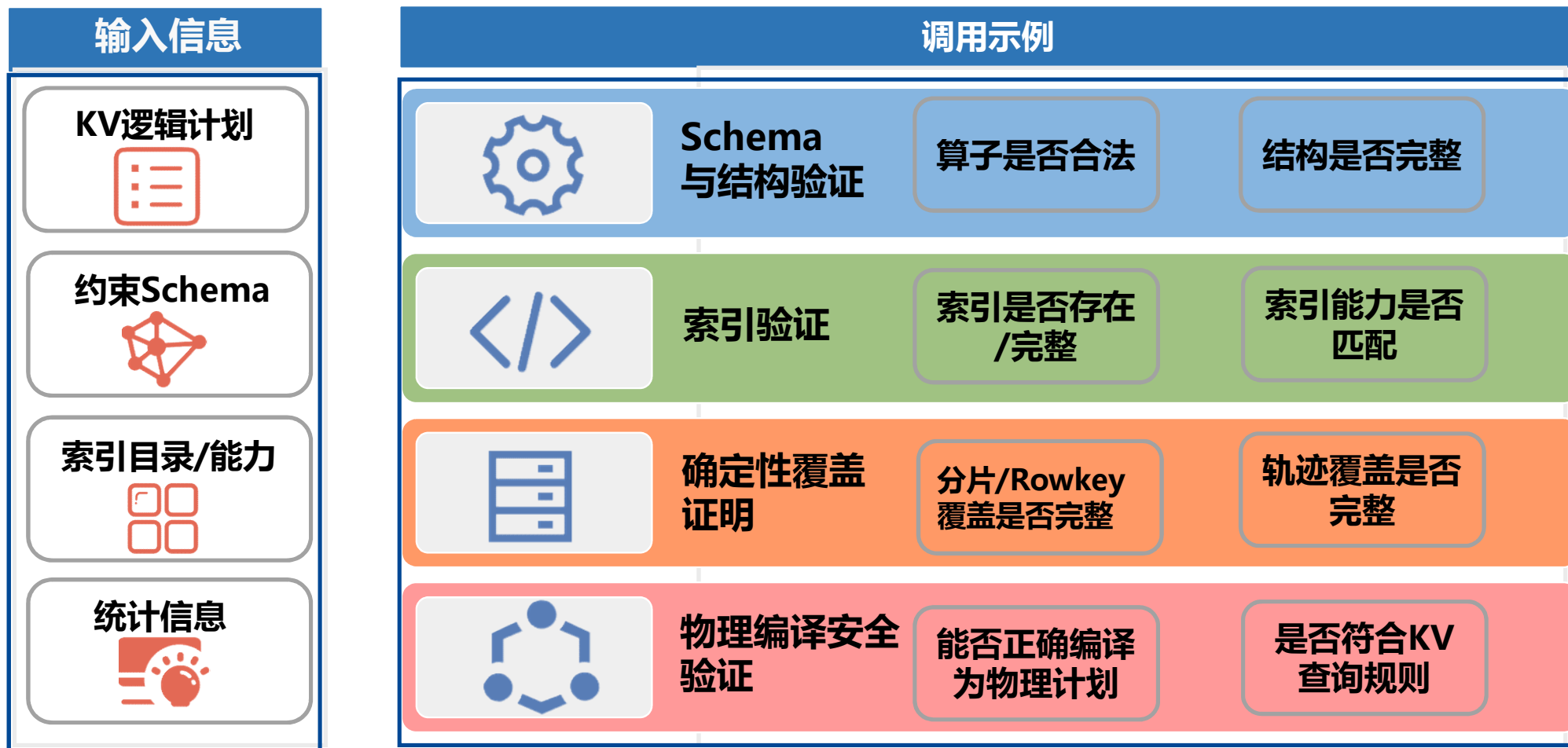
LLM自动识别问题
主动提出交互



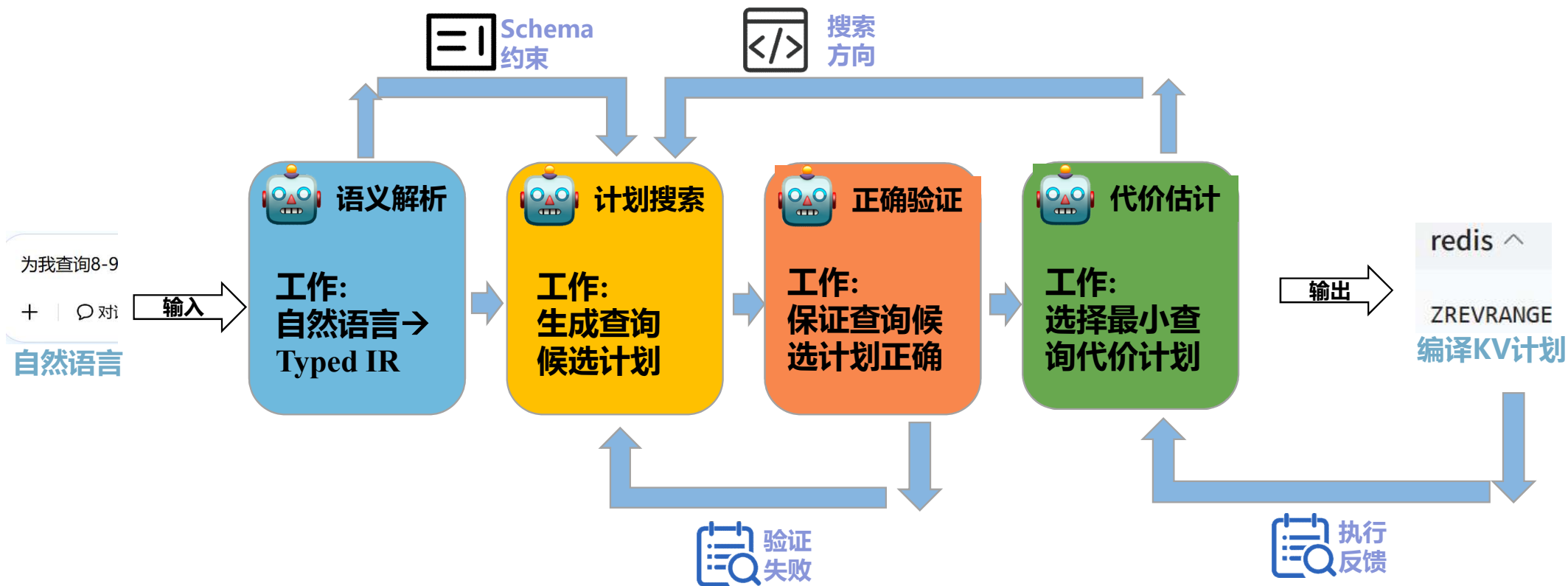
查询路径搜索机制



正确性保障机制



整体框架



系统边界



数据库

HBase



数据

二维轨迹数据

$pi=(t_i, x_i, y_i)$

T-Drive, CD-taxi

查询

- 相似轨迹 Top-K 查询
- 轨迹时空相交查询
- 轨迹空间范围查询
- 轨迹时间范围查询

索引

- Z-order
- Quadtree
- Hash

固定字段

source

Table/entity/dataset

temporal

Start/end

spatial

Geometry/null

semantics

Exist/null

similarity

DTW/Fréchet/Hausdorff

result

Top-k/trajtory_id

snapshot

Data_version/index_version

自动校验约束

类型与语义:

- 时间start < end
- 字段和索引目录存在
- 使用算子在候选集内

物理隔离:

- 禁止startRow / stopRow
- 禁止Region / Hbase命令

搜索算法



逻辑计划算子集合

访问

TimeRangeScan
ZOrderRangeScan

合并

Intersect
Union
Deduplicate

读取

BatchGetTrajectory
FetchTrajectoryChunk

精确

ExactSTFilter
Similarity
TopK

搜索流程

输入: Typed IR +
Catalog + Stats

访问模板

Schema/Coverage
剪枝

LLM提议动作

Fast Cost Evaluate
保留最优

停止条件:

- 无合法扩展
- 达到深度/时间预算
- 不可能找到优化计划

Hbase数据组织和物理查询



Hbase表与RowKey

Traj_raw_v1:

RowKey: shard | trajectory_id | chunk_id

Idx_time_v1:

RowKey: Shard | time_bucket | trajectory_id |
chunk_id

Idx_zorder_v1:

RowKey: shard | z_cell | trajectory_id | chunk_id

Idx_quad_v1:

RowKey: shard | cell | time_bucket | trajectory_id |
chunk_id

逻辑计划到物理计划

TimeRangeScan → RowKey区间 Scan(idx_time)

ZOrderRangeScan → z区间 Scan(idx_time)

FetchTrajectoryChunk → BatchGet(traj_raw)

Intersect/Deduplicate → 按trajectory_id + chunk_id
合并候选集合